



Advanced Verilog Lab Manual

www.maven-silicon.com

Maven Silicon Confidential

All the presentations, books, documents [hard copies and soft copies] labs and projects [source code] that you are using and developing as part of the training course are the proprietary work of Maven Silicon and it is fully protected under copyright and trade secret laws. You may not view, use, disclose, copy, or distribute the materials or any information except pursuant to a valid written license from Maven Silicon

Table of Contents

Lab Instructions.....	4
Lab - 1: Verification of 4x1 multiplexer.....	6
Lab - 2: Verification of DFF	7
Lab - 3: Code-coverage analysis for a coin collector.....	8
Lab - 4: Code-coverage analysis for a ALU.....	9
Lab - 5: Code-coverage analysis for an Arbiter	10

Lab Instructions

1. The recommended editor is vi or gvim editor
2. Synopsys VCS (Verilog Compile Simulator) is used here as the simulator.
3. The labs are copied inside the respective user's home directory i.e. **/home/<user_name>**
4. Here, **\$HOME** is used to represent **/home/<user_name>**
5. The following directory structure is followed for all the lab exercises:

sim/	- contains make file to run the simulation
rtl/	- contains DUT RTL code
tb/	- contains self-checking testbench
solution/	- contains solutions for testbench
6. The simulation process involves different steps such as:
 - a. Compilation
 - b. Elaboration
 - c. Simulation
7. Following are the VCS commands & options used for Batch mode simulation:
 - a. **vcs** – To compile Verilog files
 - b. **-debug_access+r** – To simulate the RTL in interactive mode
 - c. **-kdb** – To create Verdi knowledge database
 - d. **verdi** – To view waveform in interactive mode
8. Following are the VCS commands & options used for code coverage metrics:
 - a. **vcs** – To compile Verilog files
 - b. **-cm** – coverage compile time options
 - c. **-cm <coverage metrics>** – To generate coverage metrics, we will use **-cm line+tgl+cond+fsm+branch** where line – line coverage, tgl – toggle coverage, cond – condition coverage, fsm – fsm coverage, branch – branch coverage
 - d. **verdi** – To view waveform in interactive mode
9. We use the makefile to run all the above commands
10. The targets in makefile can be used for Compilation, simulation, deleting certain log files, etc.
11. Use “**make help**” to understand various targets that can be used in each lab exercise.

12. For any technical support to do the lab exercises, please reach out to us on tech_support@maven-silicon.com

Compiling design files

In the process of compilation, VCS creates a directory called `simv.vdb`, that contains code coverage data. This file is called as unified coverage database. The Unified Coverage database contains coverage data for code coverage and various aspects of coverage, such as hit counts etc. Coverage data of all the metrics (Line, Toggle, Branch, Condition, FSM) is accumulated in a single default directory called `simv.vdb`

Simulating and monitoring coverage

To monitor coverage during simulation, the command used is

```
./simv -cm <coverage metrics>    urg -dir simv.vdb
```

Viewing coverage report using Verdi

To view coverage reports in Verdi, we need to invoke the Verdi Coverage GUI by entering the **verdi** command with the coverage command-line option **-cov** and then load the coverage database.

```
verdi -cov -covdir simv.vdb
```

Lab - 1: Verification of 4x1 multiplexer

Objective : *Learning instantiation and simple stimulus generation*

Working Directory :

\$HOME/VLSI_RN/Advanced_verilog/lab1/mux4_1/tb

Source Code : mux4_1_tb.v

Instructions : The following instructions have been included in the source code. as comments. Refer to the comments in the source code and edit the source code.

- ✓ Instantiate the Design using name based port mapping.
- ✓ Write initial block for stimulus generation.
- ✓ Within initial begin
 - Initialise inputs to 0.
 - Use nested 'for' loop for generating stimulus for inputs.
 - Use \$finish task to finish the simulation at 1000ns.

Simulation Process :

- ✓ Go to the **sim** directory
- ✓ Call the target run_test to run the simulation: **make run_all**
- ✓ Observe the output.

Learning outcomes :

How to verify Multiplexer using command mode of simulation?

Lab - 2: Verification of DFF

Objective : *Learning how to write selfchecking testbenches using tasks*

Working Directory : \$HOME/VLSI_RN/Advanced_verilog/lab2/dff/tb

Source Code : tb_dff.v

Instructions : The following instructions have been included in the source code as comments. Refer to the comments in the source code and edit the source code.

- ✓ Instantiate the dff design.
- ✓ Understand the constants used for defining setup, hold time & clock period.
- ✓ Write clock generation logic for a clock with period of 100ns.
- ✓ Define the task 'sync_reset' for resetting the dff.
- ✓ Define the task 'load_d0' and 'load_d1' for loading input values.
- ✓ Within the tasks
 - Inputs are driven within Tsetup before the active posedge of the clock.
 - Outputs are sampled after the Thold time after the active posedge of the clock.

Simulation Process :

- ✓ Go to the **sim** directory
- ✓ Call the target run_test to run the simulation: **make run_all**
- ✓ Observe the output.

Learning outcomes :

Understand how to write self-checking Testbench for a DFF RTL design.

Lab - 3: Code-coverage analysis for a coin collector

Objective : *Analysis of Code Coverage for a Coin Collector design*

Working Directory :

\$HOME/VLSI_RN/Advanced_verilog/Code_coverage/lab3/vending_machine/tb

Source Code : tb_coincol1.v, tb_coincol2.v, tb_coincol3.v

Instructions : The following instructions have been included in the source code as comments.

- ✓ Understand the self-checking testbench concept used in the tasks.
- ✓ Within the tasks
 - Inputs are driven within Tsetup before the active posedge of the clock.
 - Outputs are sampled after Thold after the active posedge of the clock.
- ✓ Generate the code-coverage report for the RTL design, by simulating the testbenches one at a time.
- ✓ Edit the tb_coincol3.v file to achieve Statement, Branch, Condition, FSM state & transition coverage to be 100%.
- ✓ Compare the coverage reports generated by each TB and check which TB is a good quality TB.

Simulation Process :

- ✓ Go to the **sim** directory
- ✓ Call the target run_test to run the simulation: **make run1, make run2, make run3**
- ✓ Observe the output.

Learning outcomes :

Understand how to write self-checking Testbench for a RTL design and compare which one is a good quality Testbench.

Lab - 4: Code-coverage analysis for a ALU

Objective : *Analysis of Code Coverage for a ALU design.*

Working Directory :

\$HOME/VLSI_RN/Advanced_verilog/Code_coverage/lab4/alu/tb

Source Code : tb_alu.v

Instructions : The following instructions have been included in the source code as comments.

- ✓ Understand the self-checking testbench concept used in the tasks.
- ✓ Understand the Input & Output file operations.
- ✓ Within the tasks
 - Inputs are driven within Tsetup before the active posedge of the clock.
 - Outputs are sampled after Thold after the active posedge of the clock.
 - File read operations are used to generate inputs for the operands.
 - The actual DUT outputs are compared with the values stored in an array which is also updated through the file read operation.
- ✓ Generate the code-coverage report for the design by simulating the testbench.
- ✓ Statement, Branch, Toggle coverage has to be 100%.

Simulation Process :

- ✓ Go to the **sim** directory
- ✓ Call the target run_test to run the simulation: **make run1**
- ✓ Observe the output.

Learning outcomes :

Understand how to write self-checking Testbench for a RTL design and analyze the coverage report.

Lab - 5: Code-coverage analysis for an Arbiter

Objective : *Analysis of Code Coverage for an Arbiter design.*

Working Directory :

\$HOME/VLSI_RN/Advanced_verilog/Code_coverage/lab5/arbiter/tb

Source Code : bfm_arbiter.v, tb_userinterface.v

Instructions : The following instructions have been included in the source code as comments.

- ✓ Understand the self-checking testbench concept used in the tasks.
- ✓ In the bfm_arbiter.v file,
 - Understand how the processors are sending requests to the arbiter for accessing the shared RAM slave.
 - Understand how the processors are sending data & address to the arbiter.
- ✓ In the tb_userinterface.v file,
 - Understand how the bfm_arbiter is driven by the tb_userinterface using the bfm_command.
- ✓ Generate the code-coverage report for the design by simulating the testbench.
- ✓ FSM state coverage & transition coverage has to be 100%.

Simulation Process :

- ✓ Go to the **sim** directory
- ✓ Call the target run_test to run the simulation: **make run1**
- ✓ Observe the output.

Learning outcomes :

Understand how to write self-checking Testbench for a RTL design and analyze the coverage report.